

定点运算

一、移位运算

$$15.m = 1500.cm$$

机器用语 15 相对于小数点左移 2 位 (小数点不动)

左移 绝对值扩大

右移 绝对值缩小

在计算机中, 移位与加减配合可实现乘除运算

算术移位规则

符号位不变

正数 码制 添补代码
 原码、补码、反码 0

负数 原码 0

 补码 左移添 0, 右移添 1

 反码 1

例. 设机器数字长是 8 位 (含 1 位符号位), 写出 $A = +26$ 时, 3 种机器数左、右移 1 位和 2 位后的表示形式及对应的真值, 并分析结果的正确性.

解: $A = +26 = +11010$

则 $[A]_{\text{原}} = [A]_{\text{补}} = [A]_{\text{反}} = 0,0011010$

移位操作	机器数	对应的真值
	原码 = 补码 = 反码	
移位前	0,0011010	+26
左移 1 位	0,0110100	+52
左移 2 位	0,1101000	+104
右移 1 位	0,0001101	+13
右移 2 位	0,0000110	+6

例. 设机器数字长为8位(含1位符号位), 写出 $A = -26$ 时, 3种机器数左、右移1位和2位后的表示形式及对应的真值, 并分析结果的正确性.

解:

$$A = -26 = -11010$$

移位操作
算术移位是有符号数的移位!
移的时候符号位不动!

移位前
左移1位
左移2位
右移1位
右移2位

机器数			对应的真值		
原码	补码	反码	原码的	补码的	反码的
1,0011010	1,1100110	1,1100101	-26	-26	-26
1,0110100	1,1001100	1,1001011	-52	-52	-52
1,1101000	1,0011000	1,0010111	-104	-104	-104
1,0001101	1,1110011	1,1110010	-13	-13	-13
1,0000110	1,1111010	1,1111001	-6	-7	-6

原 = 1,0000111
补 = 1,1111010
原 = 1,0000110

$[x]_{\text{原}} = [x]_{\text{补}}_{\text{补}}$

小结:

	① 真值为正	② 负数的原码	③ 负数的补码	④ 负数的反码
左移丢1	出错	出错	正确	正确
右移丢1	影响精度	影响精度	影响精度	正确

算术移位与逻辑移位的区别:

算术移位	有符号数的移位
逻辑移位	无符号数的移位
逻辑左移	低位添0, 高位移丢
逻辑右移	高位添0, 低位移丢

例如:	01010011	10110010
逻辑左移	10100110	01011001
算术左移	00100110	1011001 (补码)

补码加/减法

1. 原码加/减法

加法规则:

先判符号位, 若相同, 绝对值相加, 结果符号不变; 若不同, 作减法, $|大| - |小|$, 结果与 $|大|$ 相同.

下面比上面多了一步, 要让计算机判断绝对值大小, 存在步调不一致的问题, 给硬件带来麻烦. 设计

减法规则:

两个原码表示的数相减, 首先将减数符号取反, 然后将被减数与符号取反后的减数按原码加法进行运算

2. 补码加法运算

$[x]_{补} + [y]_{补} = [x+y]_{补}$ $\rightarrow$ 对定点数成立, 对浮点数不成立

证明: 假设 $|x| < 1, |y| < 1, |x+y| < 1$ 以定点小数为例

① $x > 0, y > 0$, 则 $x+y > 0$

$$[x]_{补} = x, [y]_{补} = y, [x+y]_{补} = x+y = [x]_{补} + [y]_{补} \text{ 成立}$$

② $x > 0, y < 0$, 则 $x+y > 0$ 或 $x+y < 0$

$$[x]_{补} = x, [y]_{补} = 2+y, [x]_{补} + [y]_{补} = x+2+y$$

当 $x+y > 0$ 时, $x+2+y > 2$, 进位 2 必丢失 模为 2 时成立

$$\therefore [x]_{补} + [y]_{补} = x+y = [x+y]_{补} \text{ 成立}$$

当 $x+y < 0$ 时, $x+2+y < 2$,

$$\therefore [x]_{补} + [y]_{补} = x+2+y = 2+(x+y) = [x+y]_{补} \text{ 成立}$$

③ $x < 0, y > 0$ 与 ② 同理

④ $x < 0, y < 0$, 则 $x+y < 0$

$$[x]_{补} = 2+x, [y]_{补} = 2+y, [x]_{补} + [y]_{补} = 2+x+2+y = 2+(2+x+y)$$

$\underbrace{2+x+y}_{1 < 1 < 2}$

进位 2 必丢失.

$$[x]_{补} + [y]_{补} = 2+(x+y) = [x+y]_{补} \text{ 成立}$$

连同符号位一起相加, 符号位产生的进位自然丢掉

特点:

① 符号位要作为数的一部分一起参加运算

② 在模 2 意义下相加, 大于 2 的进位要丢掉 (最高位进位丢掉)

结论: 在模 2 意义下, 任意两数的补码之和 = 两数之和的补码

3. 补码减法运算

$$[x-y]_{\text{补}} = [x]_{\text{补}} + [-y]_{\text{补}}$$

证明: 假设 $|x| < 1, |y| < 1, |x+y| < 1$

① $x > 0, y > 0$, 则 $x-y > 0$ 或 $x-y < 0$

$$[x]_{\text{补}} = x, [-y]_{\text{补}} = 2-y \quad [x]_{\text{补}} + [-y]_{\text{补}} = x+2-y$$

当 $x-y > 0$ 时, $x+2-y > 2$, 进位2必丢失

$$\therefore [x]_{\text{补}} + [-y]_{\text{补}} = x-y = [x-y]_{\text{补}} \quad \text{成立}$$

当 $x-y < 0$ 时, $x+2-y < 2$

$$\therefore [x]_{\text{补}} + [-y]_{\text{补}} = x+2-y = 2+(x-y) = [x-y]_{\text{补}} \quad \text{成立}$$

② $x > 0, y < 0$, 则 $x-y > 0$

$$[x]_{\text{补}} = x, [-y]_{\text{补}} = -y$$

$$[x-y]_{\text{补}} = x-y = [x]_{\text{补}} + [-y]_{\text{补}} \quad \text{成立}$$

③ $x < 0, y > 0$ 与②同理

④ $x < 0, y < 0$, 则 $x-y > 0$ 或 $x-y < 0$

$$[x]_{\text{补}} = 2+x, [-y]_{\text{补}} = -y \quad [x]_{\text{补}} + [-y]_{\text{补}} = 2+x-y \quad \text{成立}$$

当 $x-y > 0$ 时, $2+x-y > 2$, 进位2必丢失

$$[x]_{\text{补}} + [-y]_{\text{补}} = x-y = [x-y]_{\text{补}} \quad \text{成立}$$

当 $x-y < 0$ 时, $2+x-y < 2$

$$[x]_{\text{补}} + [-y]_{\text{补}} = 2+(x-y) = [x-y]_{\text{补}} \quad \text{成立}$$

例. 设 $A=0.1011$, $B=-0.0101$, 求 $[A+B]_{补}$

解: $[A]_{\text{补}} = 0.1011$

$$[B]_{\text{补}} = 1.1011$$
$$[A]_{\text{补}} + [B]_{\text{补}} = \cancel{1}0.0110 = [A+B]_{\text{补}}$$

验证: $A+B=0.1011-0.0101=0.0110$

$$[A+B]_{补} = 0.0110$$

例. 设 $A = -9$, $B = -5$, 求 $[A+B]$ 补

解: $[A]_{\text{原}} = 1,1001$

$$[A]_{\text{补}} = 1, 0111$$

$[B]_{\text{原}} = 1, 0101$

$$[B]_{\text{补}} = 1,1011$$
$$[A+B]_{\text{补}} = [A]_{\text{补}} + [B]_{\text{补}} = 1,0111 + 1,1011 = \cancel{1},0010$$

验证: $A+B=1,1001+1,0101=1,1110$
~~即 $=1110$~~

$$A+B = -1001 + (-0101) = -1110$$

例. 设机器数字长是8位(含1位符号位)且 $A=15$, $B=24$, 用补码求 $A-B$.

解: $A = 15 = 0001111$

$[A]_{补} = 0,000111$ “变补”运算

$$B = 24 = 0011000$$

$[B]_{\text{补}} = 0,0011000 \rightarrow [-B]_{\text{补}} = 1,1101000$

$$[A-B]_{\text{补}} = [A]_{\text{补}} + [-B]_{\text{补}} = 0,0001111 + 1,1101000 = 1,1101111$$

又 $\because A-B < 0$

$$\therefore A-B = 1,000|00| = -100| = -9$$

练习. 设 $x = \frac{9}{16}$, $y = \frac{11}{16}$, 用补码求 $x+y$

解: $x = 0.1001$ $[x]_{\text{补}} = 0.1001$

$$y = 0.1011 \quad [y]_{\text{补}} = 0.1011$$
$$[x]_{\text{补}} + [y]_{\text{补}} = 0.1001 + 0.1011 = 1.0100 \\ = [x+y]_{\text{补}}$$
$$\therefore x+y = 1.1100 = -0.1100 = -\frac{12}{16} \quad (\otimes)$$

原因: 超过13, 定点小数表示不了, 造成有效数位丢失

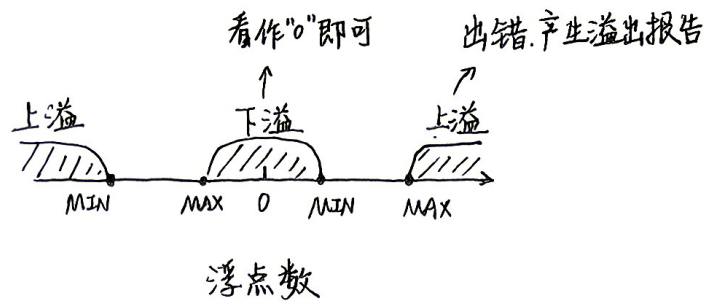
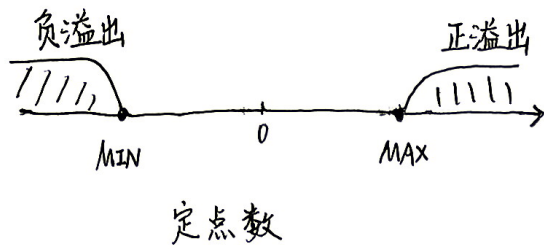
溢出

练习. 设机器数字长为8位(含1位符号位)且 $A=-97$, $B=+41$, 用补码求 $A-B$

解:

$$A = -97 = 1,1100001 \quad [A]_{\text{补}} = 1,0011111$$
$$B = +4 = 0, 0100 \quad [B]_{补} = 0, 0100 \quad [-B]_{补} = 1, 1011$$
$$\therefore [A-B]_{\text{补}} = [A]_{\text{补}} + [-B]_{\text{补}} = 1,001111 + 1,101011 = 10,110110$$
$$\therefore A-B = +1110110 = +118 \quad (\otimes)$$

溢出概念与检测方法



可能产生溢出的情况:

- ① 两正数加, 变负数, 正溢 (大于机器数所能表示的最大数)
- ② 两负数加, 变正数, 负溢 (小于机器数所能表示的最小数)

溢出判断方法:

① 一位符号位判溢出

参加操作的两个数 (减法时即为被减数和求补以后的减数) 符号相同, 其结果的符号与原操作数的符号不同, 即为溢出。

硬件实现

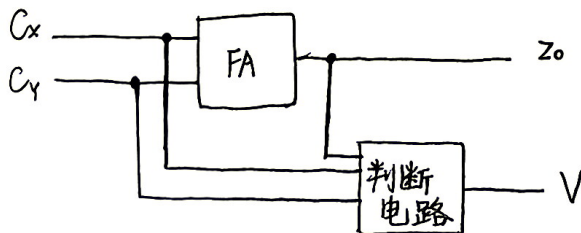
其余6种: V 为 0

C_x	C_y	Z_0	V	
0	0	1	1	2个正数相加, 结果为负数 (正溢)
1	1	0	1	2个负数相加, 结果为正数 (负溢)

$$V = m_1 + m_6 \quad (\text{最小项表达式})$$

$V=1$: 有溢出; $V=0$: 无溢出 (计算结果正确)

判断电路



② 两位符号位判溢出

双符号位补码，即补码的符号位是2位，模为4。将模扩大了2倍，不会造成有效数位的丢失

正数：补码符号位是00

负数：补码符号位是11

$$\text{变形补码 } [x]_{\text{补}}' = \begin{cases} x & 1 > x \geq 0 \\ 4+x & 0 > x \geq -1 \pmod{4} \end{cases}$$

$$[x]_{\text{补}}' + [y]_{\text{补}}' = [x+y]_{\text{补}}' \pmod{4}$$

$$[x-y]_{\text{补}}' = [x]_{\text{补}}' + [-y]_{\text{补}}'$$

结果的双符号位相同(00,11)：未溢出

结果的双符号位不同(01,10)：溢出

最高符号位代表其真正的符号

产生溢出的原因：符号位向前产生了有效的进位，但进位被丢掉了。

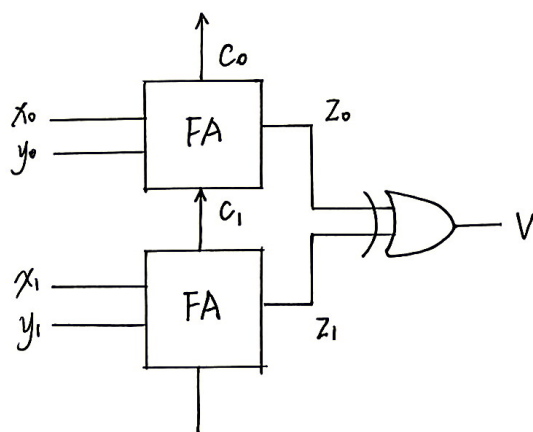
溢出是由进位引起的，但不是所有的进位都会引起溢出。

以"10"为例：产生溢出只能影响到右边这个符号位

但不影响左边的，左边符号始终代表结果的正确符号

"10"最高位是"1"，表明结果的符号是负的。

又由于"1"和"0"不同，说明产生了负溢出。



$$V = S_{f1} \oplus S_{f2}$$

例. $x = +01100$, $y = +01000$, 求 $x+y$

$$[x]_{\text{补}} = 001100$$

$$[y]_{\text{补}} = 001000$$

$$\begin{array}{r} [x]_{\text{补}} \quad 001100 \\ + [y]_{\text{补}} \quad 001000 \\ \hline \end{array}$$

$$[x+y]_{\text{补}} = 010100$$

正溢

例. $x = -11100$, $y = -10100$, 求 $x+y$

$$[x]_{\text{补}} = 110100$$

$$[y]_{\text{补}} = 111000$$

$$\begin{array}{r} [x]_{\text{补}} \quad 110100 \\ + [y]_{\text{补}} \quad 111000 \\ \hline \end{array}$$

$$[x+y]_{\text{补}} = 101100$$

负溢

⑦ 利用进位值的判别法

$$\begin{array}{r} [x]_{\text{补}} \quad x_0 \ x_1 \quad 00.1100 \\ + [y]_{\text{补}} \quad 00.1100 \\ \hline 01.1000 \end{array}$$

$$\begin{array}{r} [x]_{\text{补}} \quad 11.0100 \\ + [y]_{\text{补}} \quad 11.1000 \\ \hline 10.1100 \end{array}$$

将最高数值位向符号位的进位和符号位向前的进位作比较

都有进位

都无进位

一致，则不溢出；不一致，则溢出。

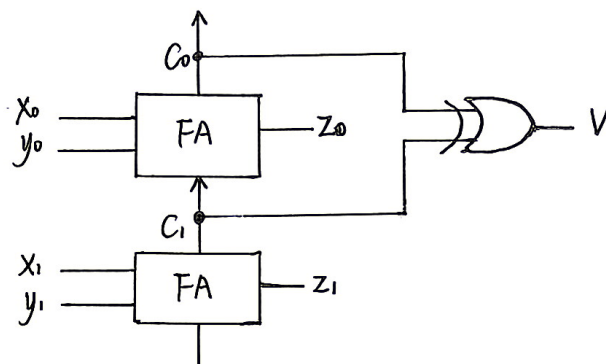
当最高有效位有进位而符号位无进位时，正溢

当最高有效位无进位而符号位有进位时，负溢

正数相加为负数或负数相加为正数，就产生溢出

溢出的逻辑表达式： $V = C_1 \oplus C_0$ (异或)

C_0 是符号位产生的进位， C_1 是最高有效位产生的进位



与双符号位法相比，进位值判别法更好：

若运算器长度固定，前者表达结果的范围只有后者的一半。

前者有2位符号位，比后者少1位数值位

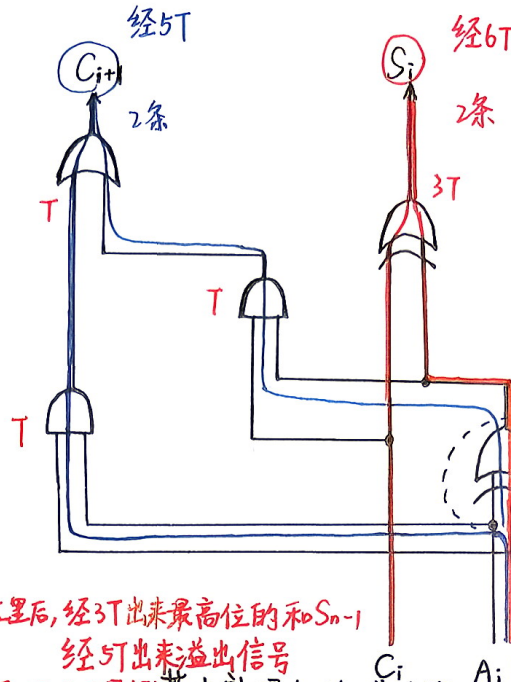
补码加减法的硬件配置

全加器

2个1位的二进制数加法

真值表:

输入			输出	
被加数	加数	来自低位的进位	本位和	向高位的进位
A_i	B_i	C_{i-1}	S_i	C_i
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



$$S_i = A \oplus B_i \oplus C_i$$

$$C_{i+1} = A_i B_i + (A_i \oplus B_i) C_i$$

信号到这里后, 经3T出来最高位的和 S_{n-1}

经5T出来溢出信号

所以用下面的(最慢)基本补码加法减法器来算.

$$M=0: [A]_{补} + [B]_{补} + (C_0=0) = [A+B]_{补} \rightarrow \text{得和的补码}$$

$$M=1: [A]_{补} + [B]_{补} + (C_0=1) = [A-B]_{补} \rightarrow \text{得差的补码}$$

$$[-B]_{补}$$

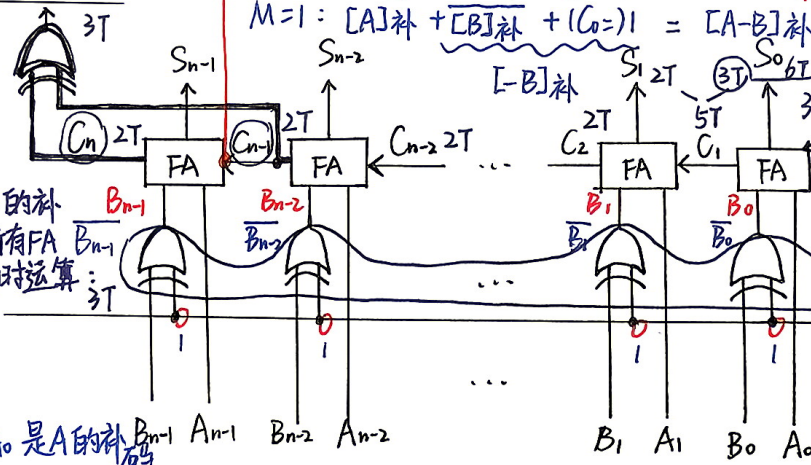
所有FA的...同时都算完了.

3T后, 开始运算.

$M=0$: 加 与0 $\oplus$, 不变
 $M=1$: 减 与1 $\oplus$, 变“反”.

2个n位的呢?

进位判别法



A, B的补码给出后, 对B的补码进行预变换处理, 所有FA的A, B输入就有异或门同时运算.

注意: $A_{n-1}A_{n-2}...A_1A_0$ 是A的补码

补码的符号位 $B_{n-1}B_{n-2}...B_1B_0$ 是B的补码符号位

n位 行波进位补码加法减法器

时间延时: 给出 $[A]_{补}$, $[B]_{补}$, M , 经多长时间运算结果出来?

设置硬件电路时间延时的基本时间单位:



在算 S_0 的时候

C_0 与 S_{n-1} 具有并行性, 只管 C_0 即可, 因为往前推进 C_0 时 S_{n-1} 已经算完了

注意: 输入信号A, B给出后, 最上图()部分的异或门就可以算了,

所以每个FA经2T就可以给出进位信号

$$3T + 5T + 2T \times (n-1) + 3T = (2n+9)T$$

位数越长, 延时越长, 速度越慢